

Introduction

Two Sum is usually the first problem anyone sees on LeetCode, and there's a reason it stays there. It looks trivial — find two numbers in a list that add up to a target — and that's exactly why it's dangerous in an interview. The obvious solution works. It's also the wrong answer.

What separates a junior response from a senior one here isn't whether you can solve Two Sum. It's whether you notice that your first solution rescans data you've already looked at, and whether you know how to stop doing that. That instinct — trade a repeated scan for a single lookup — shows up constantly: rate limiting, deduplication, caching, database indexing. Two Sum is the smallest possible version of a pattern you'll use for the rest of your career.

This walkthrough covers both approaches end to end: why the brute force costs 100 million steps on realistic input, how a hash map cuts that down to 10,000, and why the fix isn't really about hash maps at all — it's about not throwing away information you already have.

Problem Overview

You're given a list of integers and a target number. Somewhere in that list, exactly two numbers add up to the target — no more, no less, guaranteed. Your job is to return the *positions* of those two numbers, not the numbers themselves.

A few constraints matter more than they look:

- You can't use the same element twice, even if the array has duplicate values at different positions.
- Exactly one valid pair exists, so you don't need to handle "no answer" or "multiple answers."
- The order of the two returned indices doesn't matter.

That last constraint — one guaranteed pair — is a gift. It means you don't need to keep scanning after you find a match. The moment you find it, you're done.

Example

```
nums = [2, 7, 11, 15]
target = 9
```

$2 + 7 = 9$, and they sit at indices 0 and 1. The answer is `[0, 1]`.

Nothing else in the array matters — 11 and 15 are noise. The problem isn't "check everything," it's "find the one thing," and that distinction is what separates the two approaches you're about to see.

Intuition

Here's how an engineer actually arrives at the fast solution, rather than being handed it.

Say you're standing at index 0, looking at 2. You don't need to ask "what does 2 add up to with every other number?" You can flip the question: "what number, if it existed earlier in the array, would have already paired with 2?" That number is $\text{target} - 2$, or 7.

You haven't seen a 7 yet. Fine — remember that 2 is out there waiting for its partner, and keep moving.

Now you reach 7. Ask the same question: what would pair with 7 to make 9? That's 2. Have you seen a 2? Yes — you remembered it two steps ago.

That's the entire algorithm. You're never comparing every pair. You're asking, for each number, "have I already met the number that completes this one?" The moment the answer is yes, you're done.

The brute force version asks a fundamentally different, more expensive question: "for this number, check it against every other number in the array." That's checking-after — you look at a number, then go search for its partner from scratch. The fast version is checking-as-you-go — you keep a running memory, so the search is never more than one lookup.

The tool that makes "have I seen this before" a single step instead of a scan is a hash map. Think of a coat check at a theater: you don't dig through a rack of coats to find yours, you hand over a ticket and get it back instantly, because someone already indexed it. A Python dictionary does the same thing for numbers — it remembers a value and where it came from, and checking whether that value exists is a constant-time operation, not a search.

Brute Force Solution

Idea: For every number in the array, check it against every other number, and see if any pair sums to the target.

Algorithm: 1. Loop over each index i . 2. For each i , loop over every index j after it. 3. If $\text{nums}[i] + \text{nums}[j] == \text{target}$, return $[i, j]$.

Advantages: - Trivial to write, no extra data structures. - Uses $O(1)$ extra space. - Easy to reason about — you're not trusting a helper structure to be correct.

Disadvantages: - Rescans the array from scratch for every element. - Degrades badly as input grows — this is the whole point of the video's 100-million-step example.

Complexity: Time $O(n^2)$, because for every one of the n elements you may scan up to n more. Space $O(1)$, since no extra storage is used beyond the loop variables.

For a 4-element array, that's 16 comparisons — bearable. For 10,000 elements, that's roughly 100 million comparisons. The algorithm didn't change; the input did, and the cost scaled with the square of it.

Optimal Solution

The fix is to stop re-deriving information you already walked past. As you move through the array once, keep a dictionary mapping *value seen* $\rightarrow$ *index where you saw it*.

For each number, compute its complement (`target - number`). Check the dictionary:

	Step	Check dictionary for	Found?	Action
See	2	at index 0 complement 7	No	Store 2 $\rightarrow$ 0
See	7	at index 1 complement 2	Yes, at index 0	Return [0, 1]

Two things make this correct, not just fast:

1. **You check before you insert.** This is why the same element never pairs with itself — when you look for 2's complement, 2 itself isn't in the dictionary yet, because you haven't stored it. It only enters the dictionary *after* the check.
2. **You only need to look backward.** Because you're guaranteed exactly one valid pair, the first time you find a match, it's the answer. There's no need to keep scanning or to compare every element against every other.

Python Code

```
def two_sum(nums: list[int], target: int) -> list[int]:
    """Return indices of the two numbers that add up to target."""
    seen = {} # value -> index

    for index, value in enumerate(nums):
        complement = target - value
        if complement in seen:
            return [seen[complement], index]
        seen[value] = index

    # Problem guarantees a solution, so this line is unreachable.
    raise ValueError("no two sum solution found")
```

Code Walkthrough

- `seen = {}` creates the dictionary that stands in for "everything I've already looked at."
- `enumerate(nums)` gives both the value and its position in one pass, so we never need a second loop to recover an index later.
- `complement = target - value` is the reversed question from the intuition section: "what would I need to have already seen to finish this pair?"
- `if complement in seen` is the one-step lookup that replaces the brute force's inner loop. Dictionary membership checks in Python run in average $O(1)$ time.
- `return [seen[complement], index]` returns the *earlier* index first, then the current one. Order doesn't matter per the problem's rules, but this is naturally chronological.
- `seen[value] = index` only runs if no match was found — this is what keeps the current number from matching itself.

Dry Run

```
nums = [3, 2, 4], target = 6
```

Index	Value	Complement	In seen ?	Action	seen after
0	3	3	No	store	{3: 0}
1	2	4	No	store	{3: 0, 2: 1}
2	4	2	Yes, at index 1	return [1, 2]	—

At index 2, the complement of 4 is 2, and 2 is already sitting in the dictionary at index 1. Return [1, 2] immediately — no need to look at anything else.

Complexity Analysis

Time: $O(n)$. Each element is visited exactly once. The dictionary lookup and insertion are both average $O(1)$, so the total work scales linearly with the size of the input, not its square.

Space: $O(n)$. In the worst case — no match until the very last element — the dictionary holds nearly every value in the array before the answer is found.

This is a textbook time-for-space trade: you spend $O(n)$ memory to avoid $O(n^2)$ time. For anything beyond a handful of elements, that trade is almost always worth making.

Alternative Solutions

If the array were sorted, a two-pointer approach (one pointer at each end, moving inward based on whether the current sum is too high or too low) would solve this in $O(n)$ time and $O(1)$ space — no extra dictionary needed. The catch is that Two Sum doesn't guarantee sorted input, and sorting it yourself costs $O(n \log n)$ while also destroying the original indices you need to return. That's precisely

the setup for the follow-up problem, Two Sum II, where the array *is* sorted and two pointers become the right tool.

Edge Cases

- **Empty or single-element array:** No valid pair can exist; the problem's guarantee doesn't apply, so this shouldn't occur in valid input, but a defensive version should not silently return a wrong pair.
- **Duplicate values:** e.g., `[3, 3]` with target `6`. Works correctly because the current element is checked against the map *before* being inserted — the first 3 can't match itself, only the second 3 completes the pair.
- **Negative numbers:** The complement math (`target - value`) works identically whether numbers are negative, zero, or positive.
- **Target achieved by the first two elements:** The loop exits on the very first check after two stores — still $O(n)$ in the best case, just with a small constant.
- **Large duplicate value overwritten in the dictionary:** If a value appears more than twice, `seen[value] = index` overwrites the earlier index with the latest one, which only matters if multiple valid pairs exist — outside this problem's guarantees, but worth knowing if you adapt this pattern elsewhere.

Common Mistakes

1. **Checking the dictionary after inserting the current value**, which lets an element pair with itself.
2. **Using a nested loop out of habit** without recognizing that the problem allows a single pass.
3. **Sorting the array to make searching easier**, then returning indices from the sorted array instead of the original — silently wrong output.
4. **Assuming the dictionary needs to store all values upfront** in a separate pass, instead of building it incrementally during the same loop that checks for matches.
5. **Not handling duplicate values correctly**, either by overengineering a check that's unnecessary or by breaking on the case where the same value appears twice.
6. **Returning the value instead of the index** — a mismatch with what the problem actually asks for.

Interview Questions

- What if there could be multiple valid pairs — how would you return all of them?
- What if no valid pair is guaranteed — how do you signal "not found"?
- How would you solve this if the array were sorted?
- What's the three-number version (3Sum) and how does the approach change?

- Could you solve this with $O(1)$ space if you were allowed to sort the input?

Similar Problems

- **Two Sum II — Input Array Is Sorted:** Same goal, sorted input, solved with two pointers instead of a hash map.
- **3Sum:** Extends the same complement idea to three numbers, usually combining sorting with two pointers.
- **4Sum:** Further generalizes 3Sum with an additional layer of iteration.
- **Two Sum IV — Input is a BST:** Same problem shape, different underlying data structure.
- **Subarray Sum Equals K:** Uses the identical "remember what you've seen" hash map pattern, applied to running sums instead of individual values.

Key Takeaways

The nested loop isn't wrong, it's just expensive — and expense is invisible until the input gets large. The fix was never really about a hash map specifically; it was about recognizing that checking-after (search once you arrive) is strictly worse than checking-as-you-go (remember, then look up). That single reframe — trade a rescan for a lookup — is the most reusable idea in this entire problem, and it's worth internalizing far beyond Two Sum.

Watch the Video

For the full walkthrough, including a live timed comparison of both approaches and two quiz rounds to test your understanding as you go, watch the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series that works through interview and fundamentals-level coding problems the way an engineer actually reasons through them — building intuition first, writing code second, and never skipping the "why" behind an approach.

Call To Action

If this helped the problem click, the video has the full trace-through and both timed benchmarks — give it a watch and a like on YouTube, and subscribe if you want more of these. For deeper written breakdowns like this one, subscribe to the Substack. Drop a comment with which problem you'd like covered next, and share this with anyone prepping for interviews.

Quick Review

Which pattern fits 'find a pair summing to a target'?

Hash map lookup pattern — when you see 'find pair/complement matching target', think store-seen-values-in-a-map.

Time complexity of the one-pass hash map Two Sum?

$O(n)$ — single pass, each lookup and insert into the hash map is $O(1)$ average.

Space complexity of the hash map approach?

$O(n)$ — worst case stores every element in the map before finding the match.

Common bug when checking for the complement?

Checking the map before inserting current index, or accidentally matching an element with itself.

Nested-loop brute force vs hash map: what's the tradeoff?

Brute force is $O(n^2)$ time but $O(1)$ space; hash map trades memory for a single $O(n)$ pass.

Follow-up: what if the array is sorted?

Use two pointers from both ends, moving inward based on sum vs target — $O(n)$ time, $O(1)$ space.